



Flood Risk (Depth-Damage) — Test Results

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 22-March-2026

David K Kelly

CONFIDENTIAL — PROPRIETARY

Legal Notice

PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs (“MKM”).

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an “as is” basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

1 Test Results — Flood Risk (Depth-Damage) (MKM-DD-001)

Tests run on **22 March 2026 at 17:02:41**.

Table 1: Test Results Summary — Flood Risk (Depth-Damage) (MKM-DD-001)

Total Tests	128
Passed	128
Failed	0
Skipped	0
Duration	0.01s

Table 2: Detailed Test Results — Flood Risk (Depth-Damage)

Test Description	Acceptance Criteria	Result	Time
At distance = characteristic length, factor must equal $1/e \approx 0.368$.	<code>abs(f - 1.0 / math.e) < 0.001</code>	Pass	0.000s
At 5x the decay length, attenuation should be $\leq 1\%$ ($e^{-5} \approx 0.0067$).	<code>f < 0.01</code>	Pass	0.000s
Negative distance is clamped to zero; must return 1.0.	<code>compute_attenuation(-100) == 1.0</code>	Pass	0.000s
At zero distance, attenuation factor must be 1.0 (no decay).	<code>compute_attenuation(0) == 1.0</code>	Pass	0.000s
Zero decay length means instant decay; must return 0.0 for any distance $\neq 0$.	<code>compute_attenuation(100, 0) == 0.0</code>	Pass	0.000s
Every hour from 0..n must be present in output, in order.	<code>hours == list(range(len(gauge_readings)))</code>	Pass	0.000s
Flood depth must be non-negative at every time step.	<code>r['depth_m'] >= 0.0</code>	Pass	0.000s
Each hydrograph entry must contain hour, wse_m, depth_m, and flooded.	<code>'hour' in r; 'wse_m' in r (+2 more)</code>	Pass	0.000s
Empty gauge readings must return empty property hydrograph.	<code>build_property_hydrograph([], 8.0, 1.0, 0.9, 6.0, 0.3) == []</code>	Pass	0.000s
When floor level exceeds peak WSE, no time step should be flooded.	<code>r['flooded'] is False</code>	Pass	0.000s
With travel time of 5 hours, first 5 hours must show zero flood depth.	<code>r['depth_m'] == 0.0; r['flooded'] is False</code>	Pass	0.000s
Property hydrograph must have same number of time steps as gauge input.	<code>len(result) == len(gauge_readings)</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Low-elevation property at gauge location must flood at peak water level.	<code>max(peak_depths) > 0</code>	Pass	0.000s
Age 50 to 100	<code>mid.age > young</code>	Pass	0.000s
Age over 100	<code>old > young</code>	Pass	0.000s
Basic returns float	<code>isinstance(score, float); 0 < score <= 10</code>	Pass	0.000s
Construction concrete lower	<code>concrete < brick</code>	Pass	0.000s
Construction steel lowest	<code>steel < concrete</code>	Pass	0.000s
Construction timber higher	<code>timber > brick</code>	Pass	0.000s
Construction wood equals timber	<code>timber == wood</code>	Pass	0.000s
High ltv multiplier	<code>high > low</code>	Pass	0.000s
Ltv 60 to 80 multiplier	<code>mid > base</code>	Pass	0.000s
Ltv 80 to 95 multiplier	<code>mid > base</code>	Pass	0.000s
None ltv no multiplier	<code>0 < score <= 10</code>	Pass	0.000s
Unknown construction type	<code>0 < score <= 10</code>	Pass	0.000s
Unknown flood level	<code>0 < score <= 10</code>	Pass	0.000s
Very high flood capped	<code>score == 10.0</code>	Pass	0.000s
Very low flood lowest score	<code>vl < vh</code>	Pass	0.000s
Slope from 10m to 8m over 1000m should be 0.002 (2m/1000m).	<code>abs(s - 0.002) < 1e-6</code>	Pass	0.000s
Flat terrain (equal elevations) must return MIN_SLOPE, not zero.	<code>s == MIN_SLOPE</code>	Pass	0.000s
Slope must be the same regardless of flow direction (uses absolute value).	<code>s1 == s2</code>	Pass	0.000s
Zero horizontal distance must return MIN_SLOPE to avoid division by zero.	<code>t == 0.0; s == MIN_SLOPE</code>	Pass	0.000s
At zero is max	<code>RiskAssessor.calculate_distance_risk_factor == 1.0</code>	Pass	0.000s
Close band	<code>RiskAssessor.calculate_distance_risk_factor == 0.8</code>	Pass	0.000s
Far band	<code>RiskAssessor.calculate_distance_risk_factor == 0.2</code>	Pass	0.000s
Medium band	<code>RiskAssessor.calculate_distance_risk_factor == 0.4</code>	Pass	0.000s
Monotone decrease	<code>all(factors[i] >= factors[i + 1] for i in range(len(facto...</code>	Pass	0.000s
Near band	<code>RiskAssessor.calculate_distance_risk_factor == 0.6</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Negative returns default	<code>RiskAssessor.calculate_distance_risk_factor(0)</code> == 0.5	Pass	0.000s
None returns default	<code>RiskAssessor.calculate_distance_risk_factor(0)</code> == 0.5	Pass	0.000s
Very close	<code>RiskAssessor.calculate_distance_risk_factor(5)</code> == 1.0	Pass	0.000s
Very far	<code>RiskAssessor.calculate_distance_risk_factor(50)</code> == 0.1	Pass	0.000s
At high threshold	<code>RiskAssessor.assess_flood_risk_level(1.0)</code> == "High"	Pass	0.000s
At low threshold	<code>RiskAssessor.assess_flood_risk_level(0.1)</code> == "Low"	Pass	0.000s
At medium threshold	<code>RiskAssessor.assess_flood_risk_level(0.5)</code> == "Medium"	Pass	0.000s
Below low threshold	<code>RiskAssessor.assess_flood_risk_level(0.05)</code> == "Low"	Pass	0.000s
High range	<code>RiskAssessor.assess_flood_risk_level(0.75)</code> == "High"	Pass	0.000s
Medium range	<code>RiskAssessor.assess_flood_risk_level(0.3)</code> == "Medium"	Pass	0.000s
Monotone with depth	<code>levels == order</code>	Pass	0.000s
Negative returns unknown	<code>RiskAssessor.assess_flood_risk_level(-1.0)</code> == "Unknown"	Pass	0.000s
None returns unknown	<code>RiskAssessor.assess_flood_risk_level(None)</code> == "Unknown"; <code>RiskAssessor.assess_ltv_risk_level(None)</code> == "Unknown"	Pass	0.000s
Very high	<code>RiskAssessor.assess_flood_risk_level(2.5)</code> == "Very High"	Pass	0.000s
Zero is very low	<code>RiskAssessor.assess_flood_risk_level(0.0)</code> == "Very Low"	Pass	0.000s
Daily minus 15	<code>r['reliability_score'] == 80</code>	Pass	0.000s
Decommissioned	<code>r['reliability_score'] == 0;</code> <code>r['category'] == "Unreliable"</code>	Pass	0.000s
Fully operational	<code>r['reliability_score'] == 95;</code> <code>r['category'] == "Highly Reliable"</code>	Pass	0.000s
Hourly minus 5	<code>r['reliability_score'] == 90</code>	Pass	0.000s
Maintenance required	<code>r['reliability_score'] == 70;</code> <code>r['category'] == "Reliable"</code>	Pass	0.000s
Monthly minus 50	<code>r['reliability_score'] == 20;</code> <code>r['category'] == "Unreliable"</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Realtime no adjustment	<code>rt['reliability_score'] ==</code> <code>base['reliability_score']</code>	Pass	0.000s
Result keys	<code>'reliability_score' in r;</code> <code>'category'</code> <code>in r (+1 more)</code>	Pass	0.000s
Score clamped at zero	<code>r['reliability_score'] == 0</code>	Pass	0.000s
Temporarily offline	<code>r['reliability_score'] == 30;</code> <code>r['category'] == "Low Reliability"</code>	Pass	0.000s
Unknown status	<code>r['reliability_score'] == 50;</code> <code>r['category'] == "Moderately</code> <code>Reliable"</code>	Pass	0.000s
Weekly minus 30	<code>r['reliability_score'] == 65</code>	Pass	0.000s
All items are strings	<code>all(isinstance(r, str) for r in</code> <code>recs)</code>	Pass	0.000s
Deep flood extra items	<code>len(deep) > len(shallow)</code>	Pass	0.000s
High risk four or more	<code>len(recs) >= 4</code>	Pass	0.000s
High risk mentions flood insurance	<code>any("flood insurance" in r.lower()</code> <code>for r in recs)</code>	Pass	0.000s
Low risk	<code>len(recs) >= 2</code>	Pass	0.000s
Medium risk	<code>len(recs) >= 3</code>	Pass	0.000s
Very low risk	<code>len(recs) >= 1</code>	Pass	0.000s
Base rates by level	<code>result == pytest.approx(rate)</code>	Pass	0.000s
Capped at 10 pct	<code>result == pytest.approx(0.10)</code>	Pass	0.000s
Depth increases premium	<code>with_depth > no_depth</code>	Pass	0.000s
Depth multiplier formula	<code>rate == pytest.approx(expected)</code>	Pass	0.000s
Unknown level uses default	<code>result == pytest.approx(0.005)</code>	Pass	0.000s
Zero depth no effect	<code>base == zero</code>	Pass	0.000s
At high boundary	<code>RiskAssessor.assess_ltv_risk_level(0.95)</code> <code>== "High"</code>	Pass	0.000s
At low boundary	<code>RiskAssessor.assess_ltv_risk_level(0.6)</code> <code>== "Low"</code>	Pass	0.000s
At moderate boundary	<code>RiskAssessor.assess_ltv_risk_level(0.8)</code> <code>== "Moderate"</code>	Pass	0.000s
Critical ltv	<code>RiskAssessor.assess_ltv_risk_level(0.98)</code> <code>== "Critical"</code>	Pass	0.000s
High ltv	<code>RiskAssessor.assess_ltv_risk_level(0.88)</code> <code>== "High"</code>	Pass	0.000s
Low ltv	<code>RiskAssessor.assess_ltv_risk_level(0.5)</code> <code>== "Low"</code>	Pass	0.000s
Moderate ltv	<code>RiskAssessor.assess_ltv_risk_level(0.7)</code> <code>== "Moderate"</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
None returns unknown	<code>RiskAssessor.assess_flood_risk_level(None)</code> == "Unknown"; <code>RiskAssessor.assess_ltv_risk_level(None)</code> == "Unknown"	Pass	0.000s
Normalises 95 pct	<code>RiskAssessor.assess_ltv_risk_level(95)</code> == "High"	Pass	0.000s
Normalises percentage input	<code>RiskAssessor.assess_ltv_risk_level(70)</code> == "Moderate"	Pass	0.000s
Monotonicity: deeper water must produce higher velocity at constant slope.	<code>v2 > v1</code>	Pass	0.000s
Negative depth is physically invalid; must return zero velocity.	<code>compute_manning_velocity(-1.0, 0.005)</code> == 0.0	Pass	0.000s
Negative slope (downhill in opposite direction) must use absolute value.	<code>v_pos == v_neg</code>	Pass	0.000s
Manning velocity at d=1.0m, S=0.005, n=0.04 should be ~1.77 m/s.	<code>1.5 < v < 2.0</code>	Pass	0.000s
Slopes below MIN_SLOPE must be clamped to MIN_SLOPE to avoid	<code>v == v_min</code>	Pass	0.000s
Monotonicity: steeper slope must produce higher velocity at constant depth.	<code>v2 > v1</code>	Pass	0.000s
Zero depth must produce zero velocity (no flow without water).	<code>compute_manning_velocity(0.0, 0.005)</code> == 0.0	Pass	0.000s
Zero roughness coefficient causes division by zero; must return 0.	<code>compute_manning_velocity(1.0, 0.005,</code> <code>roughness=0.0) == 0.0</code>	Pass	0.000s
Elevated ltv medium flood	"Moderate Risk" in r	Pass	0.000s
High flood overrides	"High Risk" in r and "flood" in <code>r.lower()</code>	Pass	0.000s
High ltv medium flood	"High Risk" in r	Pass	0.000s
Low flood risk	"Low Risk" in r	Pass	0.000s
Medium flood default	"Moderate Risk" in r	Pass	0.000s
Minimal risk	"Minimal" in r	Pass	0.000s
Negative value critical	"Critical Risk" in r	Pass	0.000s
Negative value high	"High Risk" in r	Pass	0.000s
Negative value moderate	"Moderate Risk" in r	Pass	0.000s
None ltv no crash	<code>isinstance(r, str)</code>	Pass	0.000s
Very high flood overrides	"High Risk" in r	Pass	0.000s
Close to water increases score	<code>near['vulnerability_score'] ></code> <code>far['vulnerability_score']</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Flooded depth computed	<code>r['flood.depth'] == pytest.approx(1.0); r['vulnerability.score'] > 0</code>	Pass	0.000s
High risk recommendations present	<code>any("flood" in rec.lower() for rec in r['recommendations'])</code>	Pass	0.000s
No flood	<code>r['flood.depth'] == 0.0; r['vulnerability.score'] == 0</code>	Pass	0.000s
None elevation returns unknown	<code>r['risk.level'] == 'Unknown'; r['flood.depth'] is None (+1 more)</code>	Pass	0.000s
None flood level returns unknown	<code>r['risk.level'] == 'Unknown'</code>	Pass	0.000s
Result dict keys	<code>set(r.keys()) == {'flood.depth', 'risk.level', 'vulnerabi...</code>	Pass	0.000s
Score capped at 100	<code>r['vulnerability.score'] <= 100</code>	Pass	0.000s
Travel time for 500m at d=1.0m, S=0.005 should be well under 1 hour.	<code>0 < t < 1.0</code>	Pass	0.000s
Monotonicity: greater distance must produce longer travel time.	<code>t2 > t1</code>	Pass	0.000s
Zero depth produces zero velocity, so travel time must be infinite.	<code>t == float('inf')</code>	Pass	0.000s
Zero distance must produce zero travel time.	<code>t == 0.0; s == MIN_SLOPE</code>	Pass	0.000s
Base pct wins when higher than depth factor	<code>var == pytest.approx(350.000)</code>	Pass	0.000s
Depth 1 to 2m	<code>var == pytest.approx(500.000)</code>	Pass	0.000s
Depth half to 1m	<code>var == pytest.approx(250.000)</code>	Pass	0.000s
Depth over 2m	<code>var == pytest.approx(800.000)</code>	Pass	0.001s
Depth under half m	<code>var == pytest.approx(100.000)</code>	Pass	0.000s
Negative property value	<code>RiskAssessor.calculate_value_at_risk(-100.000, "High") ==...</code>	Pass	0.000s
Risk level percentages	<code>var == pytest.approx(1.000.000 * pct)</code>	Pass	0.000s
Zero depth uses base pct	<code>var == pytest.approx(150.000)</code>	Pass	0.000s
Zero property value	<code>RiskAssessor.calculate_value_at_risk(0, "High") == 0.0</code>	Pass	0.000s

1.1 Test Document History

Date	Version	Description	Author
05-Mar-2026	1.0	Initial test suite (29 tests)	D. Kelly
07-Mar-2026	1.1	29 test(s) added; 29 test(s) removed (total: 29)	D. Kelly
18-Mar-2026	1.2	117 test(s) added (total: 146)	D. Kelly
22-Mar-2026	1.3	18 test(s) removed (total: 128)	D. Kelly